Scheduler Affinity

Document #:	P3941R1 [Latest] [Status]
Date:	2026-01-14
Project:	Programming Language C++
Audience:	Concurrency Working Group (SG1) Library Evolution Working Group (LEWG) Library Working Group (LWG)

Reply-to:	Dietmar Kühl (Bloomberg) <dkuhl@bloomberg.net>

Contents

One important design of `std::execution::task` is that a coroutine resumes after a `co_await` on the same scheduler as the one it was executing on prior to the `co_await`. To achieve this, `task` transforms the awaited object *`obj`* using `affine_on(`*`obj`*`, `*`sched`*`)` where *`sched`* is the corresponding scheduler. There were multiple concerns raised against the specification of `affine_on` and discussed as part of P3796R1. This proposal is intended to specifically address the concerns raised relating to `task`'s scheduler affinity and in

particular `affine_on`. The gist of this proposal is impose constraints on `affine_on` to guarantee it can meet its objective at run-time.

§ 1 Change History

§ 1.1 R1

— added wording

§ 1.2 R0 Initial Revision

§ 2 Overview of Changes

There are a few NB comments raised about the way `affine_on` works:

— US 232-366: specify customisation of `affine_on` when the scheduler doesn't change.

— US 233-365: clarify `affine_on` vs. `continues_on`.

— US 234-364: remove scheduler parameter from `affine_on`.

— US 235-363: `affine_on` should not forward the stop token to the scheduling operation.

— US 236-362: specify default implementation of `affine_on`.

The discussion on `affine_on` revealed some aspects which were not quite clear previously and taking these into account points towards a better design than was previously specified:

1. To implement scheduler affinity the algorithm needs to know the scheduler on which it was started itself. The correct receiver may actually be hard to determine while building the work graph. However, this scheduler can be communicated using `get_scheduler(get_env(rcvr))` when an algorithm is `started`. This requirement is more general than just `affine_on` and is introduced by P3718R0: with this guarantee in place, `affine_on` only needs one parameter, i.e., the sender for the work to be executed.

2. The scheduler *`sched`* on which the work needs to resume has to guarantee that it is possible to resume on the correct execution agent. The implication is that scheduling work needs to be infallible, i.e., the completion signatures of `scheduler(sched)` cannot contain a `set_error_t(E)` completion signature. This requirement should be checked statically.

3. The work needs to be resumed on the correct scheduler even when the work is stopped, i.e., the scheduling operation shall be `connected` to a receiver whose environment's `get_stop_token` query yields an `unstoppable_token`. In addition, the schedule operation shall not have a `set_stopped_t()` completion signature if the environment's `get_stop_token` query yields an `unstoppable_token`. This requirement should also be checked statically.

4. When a sender knows that it will complete on the scheduler it was start on, it should be possible to customise the `affine_on` algorithm to avoid rescheduling. This customisation can be achieved by `connecting` to the result of an `affine_on` member function called on the child sender, if such a member function is present, when `connecting` an `affine_on` sender.

None of these changes really contradict any earlier design: the shape and behaviour of the `affine_on` algorithm wasn't fully fleshed out. Tightening the behaviour scheduler affinity and the `affine_on` algorithm has some implications on some other components:

1. If `affine_on` requires an infallible scheduler modelled at least `inline_scheduler`, `task_scheduler`, and `run_loop::scheduler` should be infallible (i.e., they always complete successfully with `set_value()`). `parallel_scheduler` can probably not be made infallible.
2. The scheduling semantics when changing a `task`'s scheduler using `co_await change_coroutine_scheduler(sch)` become somewhat unclear and this functionality should be removed. Similar semantics are better modelled using `co_await on(sch, nested-task)`.
3. The name `affine_on` isn't particular good and wasn't designed. It may be worth renaming the algorithms to something different.

§ 3 Discussion of Changes

§ 3.1 `affine_on` Shape

The original proposal for `task` used `continues_on` to schedule the work back on the original scheduler. This algorithm takes the work to be executed and the scheduler on which to continue as arguments. When SG1 requested that a similar but different algorithms is to be used to implement scheduler affinity, `continues_on` was just replaced by `affine_on` with the same shape but the potential to get customised differently.

The scheduler used for affinity is the scheduler communicated via the `get_scheduler` query on the receiver's environment: the scheduler argument passed to the `affine_on` algorithm would need to match the scheduler obtained from `get_scheduler` query. In the context of the `task` coroutine this scheduler can be obtained via the promise type but in general it is actually not straight forward to get hold of this scheduler because the receiver and hence its associated scheduler is only provided by `connect`. It is much more reasonable to have `affine_on` only take the work, i.e., a sender, as argument and determine the scheduler to resume on from the receiver's environment in `connect`.

Thus, instead of using

```
affine_on(sndr, sch)
```

the algorithm is used just with the sender:

```
affine_on(sndr)
```

Note that this change implies that an operation state resulting from `connecting` `affine_on` to a receiver *`rcvr`* is `started` on the execution agent associated with the scheduler obtained from `get_scheduler(get_env(rcvr))`. The same requirement is also assumed to be met when `starting` the operation state resulting from `connecting` a `task`. While it is possible to statically detect whether the query is valid and provides a scheduler it cannot be detected if the scheduler matches the execution agent on which `start` was called. P3718r0 proposes to add this exact requirement to [exec.get.scheduler].

This change addresses US 234-364 (LWG4331).

§ 3.2 Infallible Schedulers

The objective of `affine_on(sndr)` is to execute *`sndr`* and to complete on the execution agent on which the operation was `started`. Let `sch` be the scheduler obtained from `get_scheduler(get_env(rcvr))` where *`rcvr`* is the receiver used when `connecting` `affine_on(sndr)` (the discussion in this section also applies if the scheduler would be taken as a parameter, i.e., if the previous change isn't applied this discussion still applies). If `connecting` the result of `schedule(sch)` fails (i.e., `connect(schedule(sch), rcvr)` throws where *`rcvr`* is a suitable receiver), `affine_on` can avoid `starting` the main work and fail on the execution agent where it was `started`. Otherwise, if it obtained an operation state *`os`* from `connect(scheduler(sch), rcvr)`, `affine_on` would `start` its main work and would `start(os)` on the execution agent where the main work completed. If `start(os)` is always successful, `affine_on` can achieve its objective. However, if this scheduling operation fails, i.e., it completes with `set_error(e)`, or if it gets cancelled, i.e., it completes with `set_stopped()`, the execution agent on which the scheduling operation resumes is unclear and `affine_on` cannot guarantee its promise. Thus, it seems reasonable to require that a scheduler used with `affine_on` is infallible, at least when used appropriately (i.e., when providing a receiver whose associated stop token is an `unstoppable_token`).

The current working draft specifies 4 schedulers:

1. `inline_scheduler` which just completes with `set_value()` when `start()`ed, i.e., this scheduler is already infallible.
2. `task_scheduler` is a type-erased scheduler delegating to another scheduler. If the underlying scheduler is infallible, the only error case for `task_scheduler` is potential memory allocation during `connect` of its *`ts-sender`*. If `affine_on` creates an operation state for the scheduling operation during `connect`, it can guarantee that any necessary scheduling operation succeeds. Thus, this scheduler can be made infallible.
3. The `run_loop::`*`run-loop-scheduler`* is used by `run_loop`. The current specification allows the scheduling operation to fail with `set_error_t(std::exception_ptr)`. This permission allows an implementation to use `std::mutex` and `std::con-`

dition_variable whose operations may throw. It is possible to implement the logic using atomic operations which can't throw. The set_stopped() completion is only used when the receiver's stop token, i.e. the result of get_stop_token(get_env(*rcvr*)), was stopped. This receiver is controlled by affine_on, i.e., it can provide a never_stoptoken and this scheduler won't complete with set_stopped(). If the get_completion_signatures for the corresponding sender takes the environment into account, this scheduler can also be made infallible.

4. The parallel_scheduler provides an interface to a replaceable implementation of a thread pool. The current interface allows parallel_scheduler to complete with set_error_t(std::exception_ptr) as well as with set_stopped_t(). It seems unlikely that this interface can be constrained to make it infallible.

In general it seems unlikely that all schedulers can be constrained to be infallible. As a result affine_on and, by extension, task won't be usable with all schedulers if affine_on insists on using only infallible schedulers. If there are fallible schedulers, there aren't any good options for using them with a task. Note that affine_on can fail and get cancelled (due to the main work failing or getting cancelled) but affine_on can still guarantee that execution resumes on the expect execution agent when it uses an infallible scheduler.

This change addresses US 235-363 (LWG4332). This change goes beyond the actual issue and clarifies that the scheduling operation used be affine_on needs to be always successful.

§ 3.2.1 Require Infallible Schedulers For affine_on

If affine_on promises in all cases that it resumes on the original scheduler it can only work with infallible schedulers. If a users wants to use a fallible scheduler with affine_on or task the scheduler will need to be adapted. The adapted scheduler can define what it means when the underlying scheduler fails. There are conceptually only two options (the exact details may vary) on how to deal with a failed scheduling operation:

1. The user can transform the scheduling failure into a call to std::terminate.
2. The user can consider resuming on an execution agent where the adapting scheduler can schedule to infallibly (e.g., the execution agent on which operation completed) but which is different from execution agent associated with the adapted scheduler to be suitable to continue running. In that case the scheduling operation would just succeed without necessarily running on the correct execution agent. However, there is no indication that scheduling to the adapted scheduler failed and the scheduler affinity may be impacted in this failure case.

The standard library doesn't provide a way to adapt schedulers easily. However, it can certainly be done.

§ 3.2.2 Allow Fallible Schedulers For affine_on

If the scheduler used with `affine_on` is allowed to fail, `affine_on` can't guarantee that it completes on the correct scheduler in case of an error completion. It could be specified that `affine_on` completes with `set_error(rcvr, scheduling_error{e})` when the scheduling operation completes with `set_error(r, e)` to make it detectable that it didn't complete on the correct scheduler. This situation is certainly not ideal but, at least, only affects the error completion and it can be made detectable.

A use of `affine_on` which always needs to complete on a specific scheduler is still possible: in that case the user will need to make sure that the used scheduler is infallible. The main issue here is that there is no automatic static checking whether that is the case.

§ 3.2.3 Considerations On Infallible Schedulers

In an ideal world, all schedulers would be infallible. It is unclear if that is achievable. If schedulers need to be allowed to be fallible, it may be viable to require that all standard library schedulers are infallible. As outlined above that should be doable for all current schedulers except, possibly, `parallel_scheduler`. So, the proposed change is to require schedulers to be infallible when being used with `affine_on` (and, thus, being used by `task`) and to change as many of the standard C++ libraries to be infallible as possible.

If constraining `affine_on` to only infallible schedulers turns out to be too strong, the constraint can be relaxed in a future revision of the standard by explicitly opting out of that constraints, e.g., using an additional argument. For `task` to make use of it, it too would need an explicit mechanisms to indicate that its `affine_on` use should opt out of the constraint, e.g., by adding a suitable `static` member to the environment template argument.

§ 3.3 `affine_on` Customisation

Senders which don't cause the execution agent to be changed like `just` or the various queries should be able to customise `affine_on` to avoid unnecessary scheduling. Sadly, a proposal (P3206) to standardise properties which could be used to determine how a sender completes didn't make much progress, yet. An implementation can make use of similar techniques using an implementation-specific protocol. If a future standard defines a standard approach to determine the necessary properties the implementation can pick up on those.

The idea is to have `affine_on` define a `transform_sender(s)` member function which determines what sender should be returned. By default the argument is returned but if the child sender indicates that it doesn't actually change the execution agent the function would return the child sender. There are a number of senders for which this can be done:

- `just`, `just_error`, and `just_stopped`
- `read_env` and `write_env`
- `then`, `upon_error`, and `upon_stopped` if the child sender doesn't change the execution agent

The proposal is to define a `transform_sender` member which uses an implementation-specific property to determine that a sender completes on the same execution agent as the one it was started on. In addition, it is recommended that this property gets defined by the various standard library senders where it can make a difference.

This change addresses US 232-366 (LWG4329), although not in a way allowing application code to plug into this mechanism. Such an approach can be designed in a future revision of the standard.

§ 3.4 Removing `change_coroutine_scheduler`

The current working paper specifies `change_coroutine_scheduler` to change the scheduler used by the coroutine for scheduler affinity. It turns out that this use is somewhat problematic in two ways:

1. Changing the scheduler affects the coroutine until the end of the coroutine or until `change_coroutine_scheduler` is `co_await`ed again. It doesn't automatically reset. Thus, local variables constructed before `change_coroutine_scheduler(s)` was `co_await`ed were constructed on the original scheduler and are destroyed on the replaced scheduler.
2. The `task`'s execution may finish on a different than the original scheduler. To allow symmetric transfer between two `task`s each `task` needs to complete on the correct scheduler. Thus, the `task` needs to be prepared to change to the original scheduler before actually completing. To do so, it is necessary to know the original scheduler and also to have storage for the state needed to change to a different scheduler. It can't be statically detected whether `change_coroutine_scheduler(s)` is `co_await`ed in the body of a coroutine and, thus, the necessary storage and checks are needed even for `task`s which don't use `change_coroutine_scheduler`.

If there were no way to change the scheduler it would still be possible to execute using a different scheduler, although not as direct: instead of using `co_await change_coroutine_scheduler(s)` to change the scheduler used for affinity to `s` a nested `task` executing on `s` could be `co_await`ed:

```
co_await ex::starts_on(s, [](parameters)->task<T, E> { logic }(arguments));
```

Using this approach the use of the scheduler `s` is clearly limited to the nested coroutine. The scheduler affinity is fully taken care of by the use of `affine_on` when `co_await`ing work. There is no need to provide storage or checks needed for the potential of having a `task` return to the original scheduler if the scheduler isn't actually changed by a `task`.

The proposal is remove `change_coroutine_scheduler` and the possibility of changing the scheduler within a `task`. The alternative to controlling the scheduler used for affinity from within a `task` is a bit verbose. This need under the control of the coroutine is likely relatively rare. Replacing the used scheduler for an existing `task` by nesting it within `on(s, t)` or `starts_on(s, t)` is fairly straightforward.

This functionality was originally included because it is present for, at least, one of the existing libraries, although in a form which was recommended against. The existing use changes the scheduler of a coroutine when `co_await`ing the result of `schedule(s)`; this exact approach was found to be fragile and surprising and the recommendation was to provide the functionality more explicit.

This change is not associated with any national body comment. However, it is still important to do! It isn't adding any new functionality but removes a problematic way to achieve something which can be better achieved differently. If this change is not made the inherent cost of having the possibility of having `change_routine_scheduler` can't be removed later without breaking existing code.

§ 3.5 `affine_on` Default Implementation

Using the previous discussion leads to a definition of `affine_on` which is quite different from effectively just using `continues_on`:

1. The class `affine_on` should define a `transform_sender` member function which returns the child sender if this child sender indicates via an implementation specific way that it doesn't change the execution agent. It should be recommended that some of the standard library sender algorithms (see above) to indicate that they don't change the execution agent.
2. The `affine_on` algorithm should only allow to get `connected` to a receiver `r` whose scheduler `sched` obtained by `get_scheduler(get_env(r))` is infallible, i.e., `get_completion_signatures(schedule(sched), e)` with an environment `e` where `get_stop_token(e)` yields `never_stop_token` returns `completion_signatures<set_value_t()>`.
3. When `affine_on` gets `connected`, the scheduling operation state needs to be created by `connecting` the scheduler's sender to a suitable receiver to guarantee that the completion can be scheduled on the execution agent. The stop token `get_stop_token(get_env(r))` for the receiver `r` used for this `connect` shall be an `unstoppable_token`. The child sender also needs to be `connected` with a receiver which will capture the respective result upon completion and start the scheduling operation.
4. When the result operation state gets `started` it `starts` the operation state from the child operation.
5. Upon completion of the child operation the kind of completion and the parameters, if any, are stored. If this operation throws, the storage is set up to be as if `set_error(current_exception)` were called. Once the parameters are stored, the scheduling operation is started.
6. Upon completion of the scheduling operation, the appropriate completion function with the respective arguments is invoked.

This behaviour is similar to `continues_on` but is subtly different with respect to when the scheduling operation state needs to be created and that any stop token from the receiver doesn't get forwarded. In addition `affine_on` is more constrained with respect to the schedulers it supports and the shape of the algorithm is different: `affine_on` gets the scheduler to execute on from the receiver it gets `connected` to.

This change addresses US 233-365 (LWG4330) and US 236-362 (LWG; the proposed resolution in this issue is incomplete).

§ 3.6 Name Change

The name `affine_on` isn't great. It may be worth giving the algorithm a better name.

§ 4 Wording Changes

[Editor's note: Change [exec.affine.on] to use only one parameter, require an infallible scheduler from the receiver, and add a default implementation which allows customization of `affine_on` for child senders:]

1 `affine_on` adapts a sender into one that completes on a ~~specified scheduler~~receiver's scheduler. If the algorithm determines that the adapted sender already completes on the correct scheduler it can avoid any scheduling operation.

2 The name `affine_on` denotes a pipeable sender adaptor object. For a subexpression~~s `sch` and~~ `sndr`, if ~~`decltype((sch))` does not satisfy scheduler, or~~ `decltype((sndr))` does not satisfy sender, `affine_on(sndr~~`, sch`~~`)` is ill-formed.

3 Otherwise, the expression `affine_on(sndr~~`, sch`~~)` is expression-equivalent to: `transform_sender(`*`get-domain-early`*`(sndr),`
`make-sender``(affine_on, ~~sch~~env<>(), sndr))` except that `sndr` is evaluated only once.

4 The exposition-only class template *`impls-for`* ([exec.snd.expos]) is specialized for `affine_on_t` as follows:

```
namespace std::execution {
  template<>
  struct impls-for<affine_on_t> : default-impls {
    static constexpr auto get-attrs =
      [](const auto&~~data~~, const auto& child) noexcept -> decltype(auto) {
        return ~~JOIN-ENV(SCHED-ATTRS(data),FWD-ENV(~~get_env(child)~~))~~;
      };
  };
}
```

? Let `sndr` and `ev` be subexpressions such that `Sndr` is `decltype((sndr))`. If *`sender-for`*`<Sndr, affine_on_t>` is `false`, then the expression `affine_on.transform_sender(sndr, ev)` is ill-formed; otherwise, if otherwise, it is equal to:

```
auto&[_, _, child] = sndr;
using child_tag_t = tag_of_t<remove_cvref_t<decltype(child)>>;
if constexpr (requires(const child_tag_t& t){ t.affine_on(child, env); })
    return t.affine_on(child, env);
else
    return write_env(
                                    schedule_from(get_scheduler(get_env(ev)),
        write_env(std::move(child), ev)),
      JOIN-ENV(env{prop{get_stop_token, never_stop_token()}}, ev)
    );
```

[Note 1: This causes the `affine_on(sndr)` sender to become `schedule_from(sch, sndr)` when it is connected with a receiver `rcvr` whose execution domain does not customize `affine_on`, for which `get_scheduler(get_env(rcvr))` is `sch`, and `affine_on` isn't specialized for the child sender. end note]

? *Recommended Practice*: Implementations should provide `affine_on` member functions for senders which are known to resume on the scheduler where they were started. Example senders for which that is the case are `just`, `just_error`, `just_stopped`, `read_env`, and `write_env`.

5 Let *`out_sndr`* be a subexpression denoting a sender returned from `affine_on(sndr~~, sch~~)` or one equal to such, and let *`OutSndr`* be the type `decltype((`*`out_sndr`*`))`. Let *`out_rcvr`* be a subexpression denoting a receiver that has an environment of type `Env` such that `sender_in<`*`OutSndr`*`, Env>` is `true`. Let *`sch`* be the result of the expression `get_scheduler(get_env(`*`out_rcvr`*`))`. If the completion signatures of `schedule(`*`sch`*`)` contain a different completion signature than `set_value_t()` when using an environment where `get_stop_token()` returns an `unstoppable_token`, the expression `connect(`*`out_sndr`*`, `*`out_rcvr`*`)` is ill-formed. Let `op` be an lvalue referring to the operation state that results from connecting *`out_sndr`* to *`out_rcvr`*. Calling `start(`*`op`*`)` will start `sndr` on the current execution agent and execute completion operations on *`out_rcvr`* on an execution agent of the execution resource associated with ~~`sch`~~*`sch`*. If the current execution resource is the same as the execution resource associated with ~~`sch`~~*`sch`*, the completion operation on *`out_rcvr`* may be called before `start(`*`op`*`)` completes. ~~If scheduling onto `sch` fails, an error completion on *`out_rcvr`* shall be executed on an unspecified execution agent.~~

[Editor's note: Remove `change_coroutine_scheduler` from [execution.syn]:]

```
namespace std::execution {
  ...
  // [exec.task.scheduler], task scheduler
  class task_scheduler;

  template<class E>
  struct with_error {
    using type = remove_cvref_t<E>;
    type error;
  };
  template<class E>
    with_error(E) -> with_error<E>;
```

```
  template<scheduler Sch>
  struct change_coroutine_scheduler {
    using type = remove_cvref_t<Sch>;
    type scheduler;
  };
  template<scheduler Sch>
    change_coroutine_scheduler(Sch) -> change_coroutine_scheduler<Sch>;
```

```
  // [exec.task], class template task
  template<class T, class Environment>
    class task;
  ...
}
```

[Editor's note: Adjust the use of `affine_on` and remove `change_coroutine_scheduler` from [task.promise]:]

```
namespace std::execution {
  template<class T, class Environment>
  class task<T, Environment>::promise_type {
  public:
    ...

    template<class A>
      auto await_transform(A&& a);
```

```
    template<class Sch>
      auto await_transform(change_coroutine_scheduler<Sch> sch);
```

```

    unspecified get_env() const noexcept;

    ...
  }
};
```

…

```
template<sender Sender>
  auto await_transform(Sender&& sndr) noexcept;
```

9 *Returns*: If `same_as<inline_scheduler, scheduler_type>` is `true` returns `as_awaitable(std::forward<Sender>(sndr), *this)`; otherwise returns `as_awaitable(affine_on(std::forward<Sender>(sndr)~~, SCHED(*this)~~), *this)`.

```
template<class Sch>
  auto await_transform(change_coroutine_scheduler<Sch> sch) noexcept;
```

10 *Effects*: Equivalent to:

```
return                          await_transform(just(exchange(SCHED(*this),
        scheduler_type(sch.scheduler))), *this);
```

```
void unhandled_exception();
```

11 *Effects*: If the signature `set_error_t(exception_ptr)` is not an element of `error_types`, calls `terminate()` ([except.terminate]). Otherwise, stores `current_exception()` into *`errors`*.

…

[Editor's note: In [exec.task.scheduler] change the constructor of `task_scheduler` to require that the scheduler passed is infallible]

```
template<class Sch, class Allocator = allocator<void>>
         requires(!same_as<task_scheduler,     remove_cvref_t<Sch>>)     &&
        scheduler<Sch>
explicit task_scheduler(Sch&& sch, Allocator alloc = {});
```

? *Mandates*: Let `e` be an environment and let `E` be `decltype(e)`. If `unstoppable_token<decltype(get_stop_token(e))>` is `true`, then the type `completion_signatures_of_t<decltype(schedule(sch)), E>` only includes `set_value_t()`, otherwise it may additionally include `set_stopped_t()`.

2 *Effects*: Initialize *`sch_`* with `allocate_shared<remove_cvref_t<Sch>>(alloc, std::forward<Sch>(sch))`.

3 *Recommended practice*: Implementations should avoid the use of dynamically allocated memory for small scheduler objects.

4 *Remarks*: Any allocations performed by construction of *`ts-sender`* or *`state`* objects resulting from calls on `*this` are performed using a copy of `alloc`.

[Editor's note: In [exec.task.scheduler] change the *`ts-sender`* completion signatures to indicate that `task_scheduler` is infallible:]

8
```
namespace std::execution {
  class task_scheduler::ts-sender {     // exposition only
  public:
    using sender_concept = sender_t;

    template<receiver Rcvr>
      state<Rcvr> connect(Rcvr&& rcvr) &&;
  };
}
```

`ts-sender` is an exposition-only class that models `sender` ([exec.snd]) and for which `completion_signatures_of_t<ts-sender, E>` denotes~~:~~ `completion_signatures<set_value_t()>` if `unstoppable_token<decltype(get_stop_token(declval<E>()))>` is `true`, and otherwise `completion_signatures<set_value_t(), set_stopped_t()>`.

```
completion_signatures<
  set_value_t(),
  set_error_t(error_code),
  set_error_t(exception_ptr),
  set_stopped_t()>
```

[Editor's note: In [exec.run.loop.types] change the paragraph defining the completion signatures:]

…

```
class run-loop-sender;
```

5 *run-loop-sender* is an exposition-only type that satisfies `sender`. Let `E` be the type of an environment. If `unstoppable_token<decltype(get_stop_token(declval<E>()))>` is `true`, then `completion_signatures_of_t<`*run-loop-sender*`, E>` is

```
        completion_signatures<set_value_t(),    set_error_t(exception_ptr),
         set_stopped_t()>`
```

```
  completion_signatures<set_value_t()>
```

Otherwise it is

```
  completion_signatures<set_value_t(), set_stopped_t()>
```

6 An instance of *run-loop-sender* remains valid until the end of the lifetime of its associated `run_loop` instance.

…